

بخش نظری:

الف) TinyML و استقرار مدل روی لبه (Edge Inference)

TinyML به مجموعه‌ای از روش‌ها و فناوری‌ها گفته می‌شود که امکان اجرای مدل‌های یادگیری ماشین را روی دستگاه‌های کوچک و کم‌مصرف مانند میکروکنترلرها، حسگرهای هوشمند و گره‌های اینترنت اشیا فراهم می‌کند. در این رویکرد، مدل پس از آموزش در محیطی قدرتمندتر، به نسخه‌ای سبک و بهینه تبدیل شده و مستقیماً روی دستگاه اجرا می‌شود. منظور از Edge Inference این است که عملیات پیش‌بینی (Inference) به جای ارسال داده‌ها به سرورهای ابری، روی خود دستگاه یا در نزدیک‌ترین گره شبکه انجام شود. به عنوان مثال، در این تمرین داده‌های حسگر حرکتی می‌توانند مستقیماً توسط مدل مستقر شده روی دستگاه تحلیل شوند و وضعیت حرکت (Idle)، Move یا Shake بدون نیاز به ارسال مداوم داده‌ها به ابر تشخیص داده شود. مهم‌ترین مزایای Edge Inference نسبت به پردازش ابری عبارتند از:

1. کاهش تأخیر (Latency): تصمیم‌گیری به صورت محلی انجام می‌شود و نیازی به انتظار برای پاسخ سرور نیست.
 2. کاهش مصرف پهنای باند: به جای ارسال حجم زیادی از داده‌های خام، تنها نتیجه نهایی منتقل می‌شود.
 3. افزایش حریم خصوصی و امنیت: داده‌های حساس روی دستگاه باقی می‌مانند.
 4. قابلیت کار در شرایط قطع اینترنت: مدل حتی بدون اتصال دائمی به شبکه نیز قادر به فعالیت است.
 5. کاهش مصرف انرژی و هزینه‌های زیرساخت ابری: تعداد تبادلات شبکه و بار پردازشی سرور کاهش می‌یابد.
- بنابراین TinyML و Edge Inference از فناوری‌های کلیدی در سامانه‌های AIoT محسوب می‌شوند و امکان هوشمندسازی تجهیزات کم‌مصرف را فراهم می‌کنند.

ب) مراحل چرخه عمر یک کاربرد یادگیری ماشین در اینترنت اشیا

چرخه عمر یک سامانه یادگیری ماشین در اینترنت اشیا از چند مرحله اصلی تشکیل می‌شود:

1 جمع‌آوری داده (Data Collection)

در این مرحله داده‌های مورد نیاز از حسگرها، تجهیزات IoT یا محیط شبیه‌سازی جمع‌آوری می‌شوند. کیفیت داده‌های جمع‌آوری شده تأثیر مستقیمی بر عملکرد مدل دارد.

2 پیش‌پردازش و برچسب‌گذاری داده‌ها

داده‌های خام معمولاً دارای نویز یا مقادیر نامعتبر هستند. در این مرحله داده‌ها پاک‌سازی، نرمال‌سازی و برچسب‌گذاری می‌شوند. در پروژه حاضر داده‌ها با برچسب‌های Idle، Move و Shake ذخیره شدند.

3 استخراج ویژگی‌ها (Feature Extraction)

ویژگی‌های مهم از داده‌های خام استخراج می‌شوند تا مدل بتواند الگوهای موجود را بهتر تشخیص دهد. در Edge Impulse از Spectral Features برای استخراج ویژگی‌های حرکتی استفاده شد.

4 آموزش مدل (Training)

در این مرحله الگوریتم یادگیری ماشین با استفاده از داده‌های آموزشی آموزش می‌بیند و الگوهای موجود در داده‌ها را فرا می‌گیرد.

5 ارزیابی مدل (Evaluation)

عملکرد مدل با استفاده از معیارهایی مانند Accuracy، Precision، Recall و Confusion Matrix بررسی می‌شود تا میزان دقت آن مشخص گردد.

6 فشردسازی و کمی سازی (Compression & Quantization)

برای اجرای مدل روی سخت افزارهای کم منبع، اندازه مدل کاهش داده می شود. در کمی سازی، وزن ها از قالب های پرحجم مانند Float32 به قالب هایی مانند Int8 تبدیل می شوند تا حافظه و توان پردازشی کمتری مصرف شود.

7 استقرار (Deployment)

مدل به محیط عملیاتی منتقل می شود و روی میکروکنترلر، دستگاه لبه یا سیستم هدف اجرا می گردد.

8 پایش و نگهداری (Monitoring)

پس از استقرار، عملکرد مدل در محیط واقعی بررسی می شود. در صورت کاهش دقت یا تغییر شرایط محیطی، مدل باید مجدداً آموزش داده شود.

در پروژه حاضر این مراحل با استفاده از Velxio، Node-RED، Edge Impulse و ThingsBoard پیاده سازی شدند.

ج) تفاوت طبقه بندی و تشخیص ناهنجاری

طبقه بندی (Classification) و تشخیص ناهنجاری (Anomaly Detection) دو مسئله متفاوت در یادگیری ماشین هستند. در طبقه بندی، مدل از قبل مجموعه ای از کلاس های مشخص را می شناسد و تلاش می کند هر نمونه جدید را در یکی از این کلاس ها قرار دهد. برای این کار لازم است داده های آموزشی برچسب دار برای هر کلاس در اختیار مدل قرار گیرد. در مقابل، تشخیص ناهنجاری به دنبال شناسایی نمونه هایی است که با رفتار عادی سیستم تفاوت دارند. در این روش معمولاً مدل فقط رفتار عادی را یاد می گیرد و هر داده ای که شباهت کافی به الگوهای شناخته شده نداشته باشد به عنوان ناهنجاری تشخیص داده می شود. در این تمرین هدف تشخیص سه وضعیت مشخص حرکتی شامل Idle، Move و Shake بود. از آنجا که:

- کلاس های هدف از ابتدا مشخص بودند؛
- برای هر کلاس داده آموزشی جمع آوری شد؛
- هدف تعیین نوع حرکت بود نه یافتن رفتار غیرعادی؛

بنابراین مسئله حاضر یک مسئله طبقه بندی چندکلاسه (Multi-Class Classification) محسوب می شود و استفاده از روش های طبقه بندی نسبت به تشخیص ناهنجاری مناسب تر و دقیق تر است.

د) عوامل کاهش دقت مدل در زمان اجرا و راهکارهای پیشنهادی

در سامانه های AIoT عوامل مختلفی می توانند باعث کاهش دقت مدل پس از استقرار شوند.

1 نویز و خطای حسگر

داده های حسگر ممکن است تحت تأثیر لرزش، تداخل الکترومغناطیسی یا خطاهای سخت افزاری قرار گیرند و مقادیر نادرست تولید کنند. راهکار:

استفاده از فیلترهای نرم افزاری مانند Moving Average یا Kalman Filter و همچنین کالیبراسیون دوره ای حسگر.

2 تغییر توزیع داده ها (Data Drift)

ممکن است داده های واقعی محیط با داده هایی که مدل بر اساس آن ها آموزش دیده متفاوت شوند. در نتیجه مدل به تدریج دقت خود را از دست می دهد.

راهکار:

جمع آوری داده های جدید و بازآموزی دوره ای مدل با داده های به روز.

3 عدم توازن داده‌های آموزشی

اگر برخی کلاس‌ها تعداد نمونه بسیار بیشتری نسبت به سایر کلاس‌ها داشته باشند، مدل نسبت به آن کلاس‌ها سوگیری پیدا می‌کند. راهکار:

استفاده از داده‌های متوازن برای تمام کلاس‌ها و افزایش داده‌های کلاس‌های کم‌نمونه.

4 خطاهای کمی‌سازی و فشرده‌سازی

کوچک‌سازی بیش از حد مدل ممکن است بخشی از اطلاعات مهم را حذف کرده و دقت مدل را کاهش دهد. راهکار:

مقایسه نسخه‌های مختلف مدل پس از Quantization و انتخاب بهترین نقطه تعادل بین دقت و اندازه مدل.

5 خطا در انتقال داده‌ها

از دست رفتن بسته‌های MQTT یا دریافت ناقص داده‌ها می‌تواند باعث ایجاد ورودی‌های نادرست برای مدل شود. راهکار:

استفاده از QoS مناسب در MQTT، اعتبارسنجی داده‌ها و بررسی صحت ارتباط شبکه.

منابع

1. [Edge Impulse Documentation, TinyML and Embedded Machine Learning Documentation.](#)
2. Warden, P., & Situnayake, D. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, O'Reilly Media, 2020.

بخش عملی:

مشخصات پروژه

در این پروژه یک سامانه هوشمند تشخیص وضعیت حرکتی مبتنی بر AIoT طراحی و پیاده‌سازی شد. هدف پروژه، شبیه‌سازی داده‌های حسگر حرکتی MPU6050، جمع‌آوری داده‌ها، آموزش یک مدل سبک یادگیری ماشین در Edge Impulse، استقرار مدل در لبه (Edge) و نمایش نتایج در ThingsBoard بود.

در این سامانه سه وضعیت حرکتی مختلف تعریف شد:

- Idle (ساکن)
- Move (حرکت معمولی)
- Shake (لرزش یا حرکت شدید)

پس از آموزش مدل، وضعیت حرکتی به صورت برخط تشخیص داده شده و نتیجه در داشبورد ThingsBoard نمایش داده می‌شود. همچنین در صورت تشخیص وضعیت Shake هشدار تولید می‌گردد.

1- معماری کلی سامانه

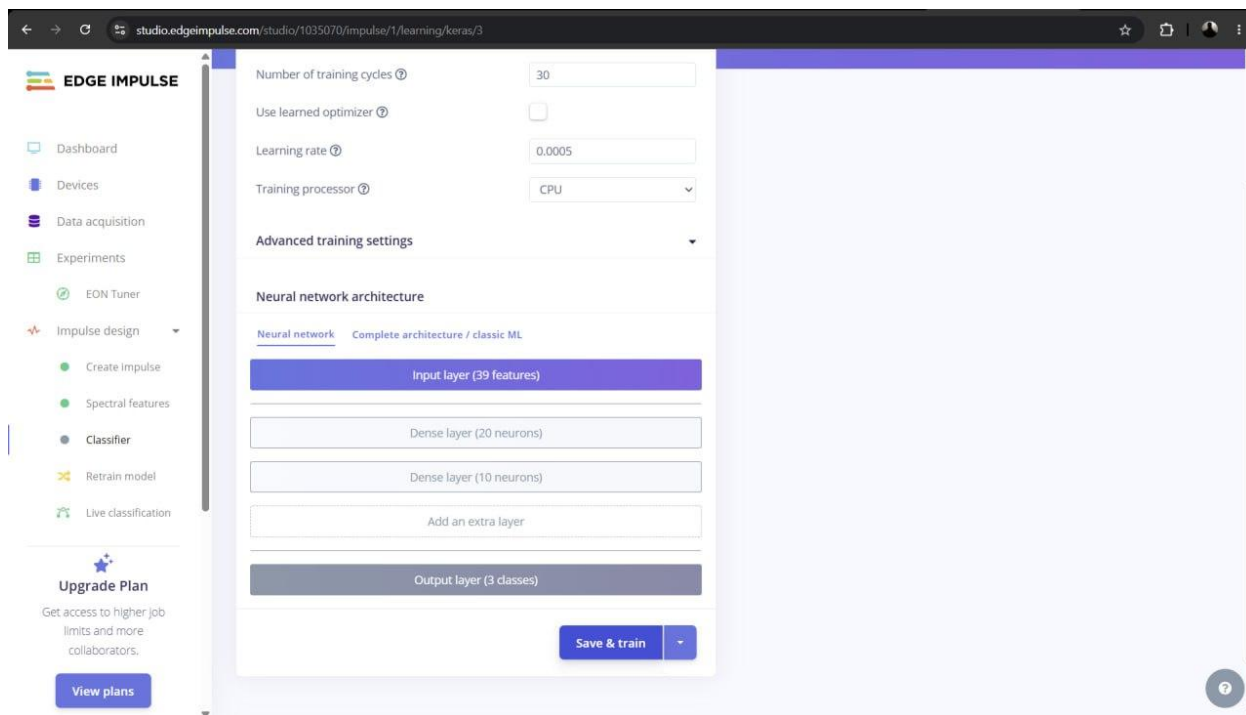
معماری پروژه از چند بخش اصلی تشکیل شده است:

1. شبیه سازی حسگر MPU6050 در Velxio
2. تولید داده های شتاب سنج سه محوره
3. ارسال داده ها از طریق MQTT
4. دریافت و پردازش داده ها در Node-RED
5. ارسال داده های آموزشی به Edge Impulse
6. آموزش مدل TinyML
7. استقرار مدل و انجام Inference
8. ارسال خروجی مدل به ThingsBoard
9. نمایش وضعیت و هشدارها در داشبورد

2- راه اندازی پروژه در Edge Impulse

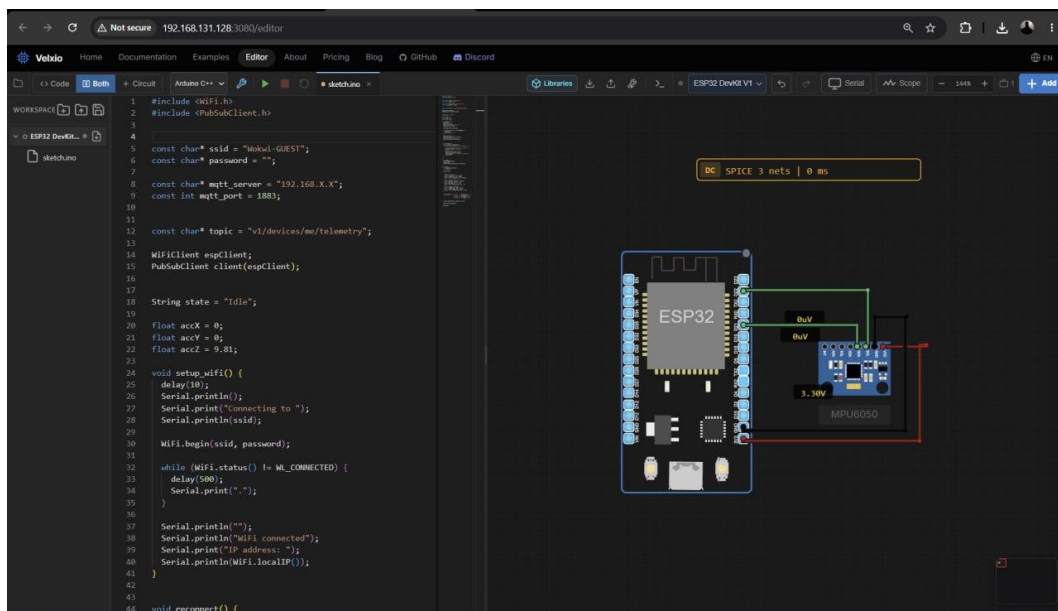
در ابتدا یک پروژه جدید در Edge Impulse ایجاد شد. این پروژه به منظور آموزش مدل تشخیص حرکت مبتنی بر داده های شتاب سنج طراحی گردید.

پس از ورود به محیط Edge Impulse ، پروژه جدید ایجاد شد و ساختار اولیه شامل بخش های Data Acquisition ، Impulse Design ، Training و Deployment آماده گردید.

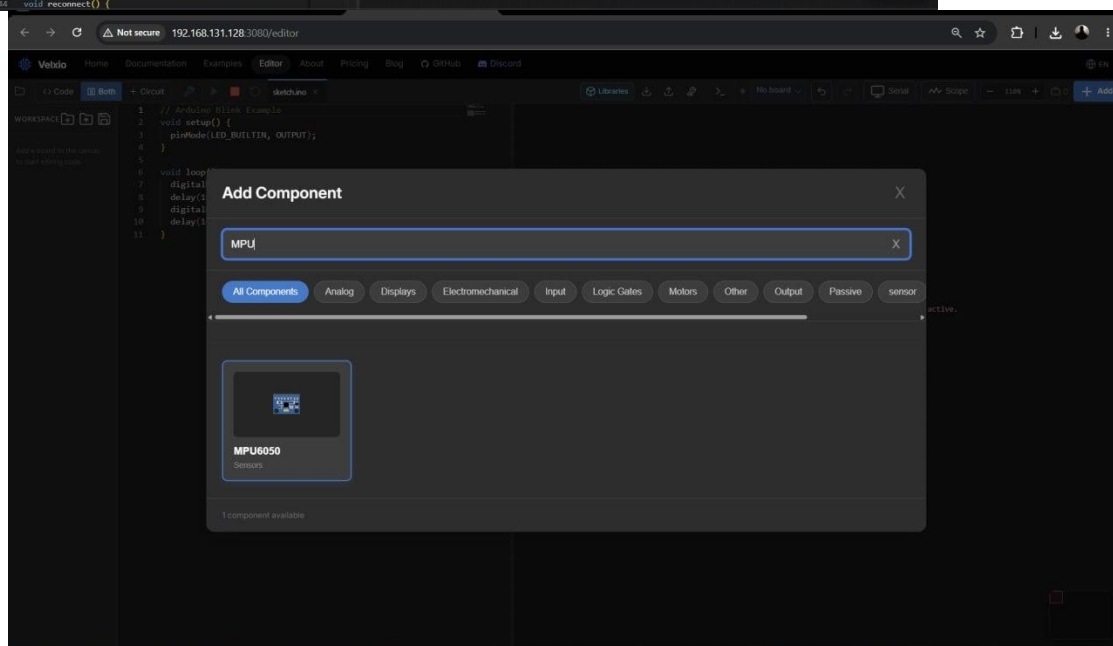


3-طراحی و شبیه‌سازی حسگر در Velxio

برای تولید داده‌های حرکتی از محیط Velxio استفاده شد. در این محیط یک برد ESP32 به همراه حسگر MPU6050 شبیه‌سازی گردید. اتصالات حسگر به صورت زیر انجام شد:



MPU6050	ESP32
VCC	3.3V
GND	GND
SDA	GPIO21
SCL	GPIO22



4-تعریف کلاس‌های حرکتی

برای آموزش مدل سه کلاس مختلف تعریف گردید:

کلاس Idle

در این وضعیت سنسور تقریباً ثابت بوده و تنها نویزهای بسیار کوچک مشاهده می‌شود.

ویژگی‌ها:

- تغییرات بسیار کم در محورها
- دامنه نوسان پایین
- رفتار نزدیک به سکون کامل

کلاس Move

در این حالت حرکت عادی و نرم شبیه‌سازی شد.

ویژگی‌ها:

- نوسانات سینوسی
- تغییرات پیوسته
- دامنه متوسط

کلاس Shake

این وضعیت بیانگر لرزش شدید و حرکات ناگهانی است.

ویژگی‌ها:

- تغییرات سریع
- دامنه بالا
- فرکانس زیاد

5-جمع‌آوری داده‌ها

برای آموزش مدل لازم بود داده‌های هر سه کلاس به صورت متوازن جمع‌آوری شوند.

پارامترهای نمونه‌برداری:

- نرخ نمونه‌برداری: 50Hz
- فاصله زمانی نمونه‌ها: 20ms
- طول پنجره: 2 ثانیه
- تعداد نمونه در هر پنجره: 100 نمونه

داده‌های تولید شده توسط ESP32 در قالب JSON منتشر شدند.

```
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3
4 const char* ssid = "Wokwi-GUEST";
5 const char* password = "";
6
7 const char* mqtt_server = "broker.hivemq.com";
8 const int mqtt_port = 1883;
9
10 const char* topic = "mohsen/velxio/telemetry";
11
12 WiFiClient espClient;
13 PubSubClient client(espClient);
14
15 String state = "Idle";
16
17 float accX = 0;
18 float accY = 0;
19 float accZ = 9.81;
20
21
22 void setup_wifi() {
23   delay(10);
24   Serial.println();
25   Serial.print("Connecting to ");
26   Serial.println(ssid);
27
28   WiFi.begin(ssid, password);
29
30   while (WiFi.status() != WL_CONNECTED) {
31     delay(500);
32     Serial.print(".");
33   }
34
35   Serial.println("");
36   Serial.println("WiFi connected");
37   Serial.print("IP address: ");
38   Serial.println(WiFi.localIP());
39 }
40
41 void reconnect() {
42   while (!client.connected()) {
43     Serial.print("Attempting MQTT connection...");
44
45     if (client.connect("ESP32_Velxio_Client")) {
46       Serial.println("connected");
47     } else {
48       Serial.print("failed, rc=");
49       Serial.print(client.state());
50       Serial.println(" try again in 2 seconds");
51       delay(2000);
52     }
53   }
54 }
55
56 void setup() {
57   Serial.begin(115200);
58   setup_wifi();
59   client.setServer(mqtt_server, mqtt_port);
60 }
61
62 void loop() {
63   if (!client.connected()) {
64     reconnect();
65   }
66   client.loop();
67
68   if (state == "Idle") {
69     accX = (random(-10, 10) / 100.0);
70     accY = (random(-10, 10) / 100.0);
71     accZ = 9.81 + (random(-10, 10) / 100.0);
72   } else if (state == "Move") {
73     float time_sec = millis() / 1000.0;
74     accX = sin(time_sec * 3) * 3.0;
75     accY = cos(time_sec * 3) * 3.0;
76     accZ = 9.81 + sin(time_sec * 5) * 1.5;
77   } else if (state == "Shake") {
78     accX = (random(-300, 300) / 10.0);
79     accY = (random(-300, 300) / 10.0);
80     accZ = 9.81 + (random(-300, 300) / 10.0);
81   }
82
83   String payload = "{\"accX\": " + String(accX, 2) +
84     ", \"accY\": " + String(accY, 2) +
85     ", \"accZ\": " + String(accZ, 2) +
86     ", \"state\": \"" + state + "\"}";
87
88   client.publish(topic, payload.c_str());
89
90   Serial.println(payload);
91   delay(20);
92 }
93
94
95
96
97
98
99 }
```

نمونه داده:

```
{
  "accX": 1.25,
  "accY": -0.54,
  "accZ": 9.81,
  "state": "Move"
}
```

6-انتقال داده‌ها با Node-RED

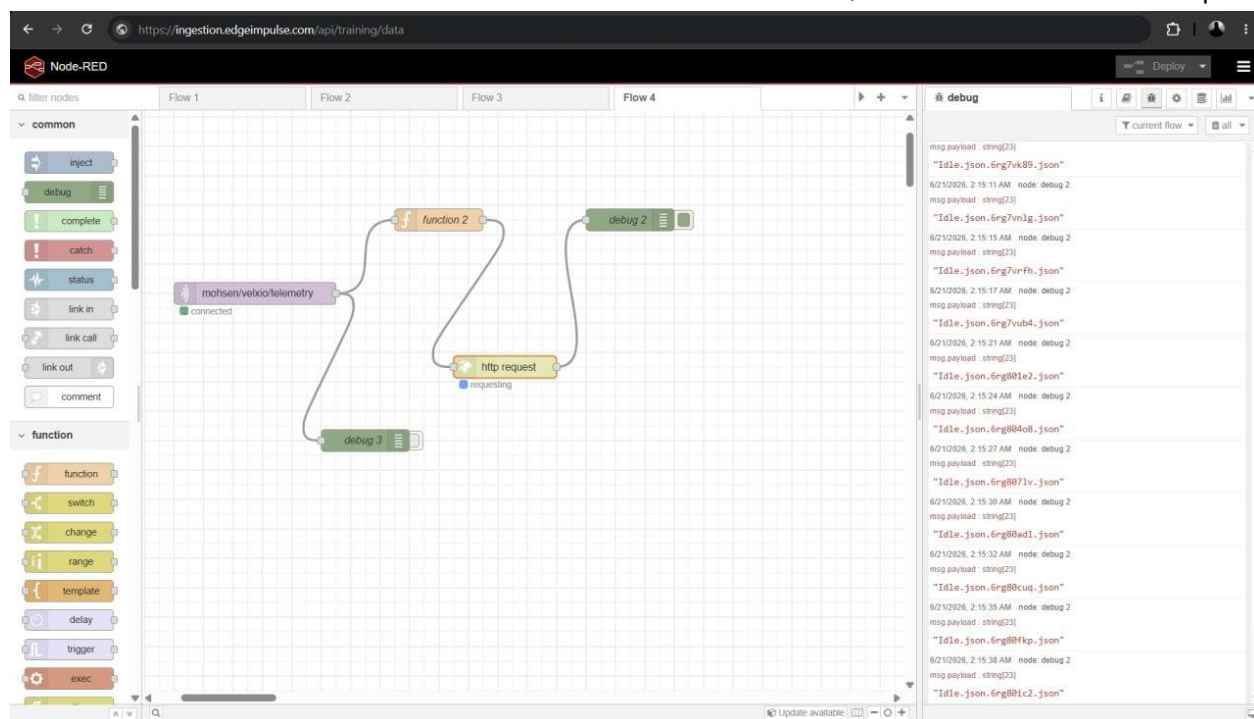
برای انتقال داده‌ها از Node-RED استفاده شد.

Node-RED وظایف زیر را بر عهده داشت:

- دریافت داده از MQTT
- جمع‌آوری نمونه‌ها
- تشکیل پنجره‌های زمانی
- آماده‌سازی فرمت مورد نیاز Edge Impulse
- ارسال داده‌ها به API آموزشی Edge Impulse

ساختار Flow به صورت زیر بود:

MQTT In → Function → HTTP Request



7-برچسب‌گذاری و مدیریت داده‌ها در Edge Impulse

پس از ارسال داده‌ها، نمونه‌ها در بخش Data Acquisition ظاهر شدند.

برای هر پنجره داده یکی از برچسب‌های زیر ثبت گردید:

- Idle
- Move
- Shake

داده‌ها به صورت متوازن جمع‌آوری شدند تا مدل دچار Bias نشود.

Configure your target device and application budget

Target device
Define your target device requirements to inform model optimizations and performance calculations. No device yet? Use the default settings which you can change at any time.

Target device: Espressif ESP-EYE (ESP32 240MHz)

Processor family: ESP32

Clock rate: 240 / MHz

Custom device name (optional):

Application budget
Specify the available RAM and ROM for the model's operation, along with the maximum allowed latency for your specific application. Not sure yet? Start with the defaults and modify them later on.

RAM: 4 / MB

ROM: 4 / MB

Buttons: Reset to default settings Cancel Save

EDGE IMPULSE

Dataset | Data explorer | Data sources | Synthetic data | AI Labeling | CSV Wizard

DATA COLLECTED: 18m 18s

TRAIN / TEST SPLIT: 80% / 20%

Dataset

SAMPLE NAME	LABEL	ADDED	LENGTH
Shake.6rg984u0	Shake	Today, 02:37:14	2s
Shake.6rg981q1	Shake	Today, 02:37:11	2s
Shake.6rg97v35	Shake	Today, 02:37:08	2s
Shake.6rg97scj	Shake	Today, 02:37:06	2s
Shake.6rg97pk2	Shake	Today, 02:37:03	2s
Shake.6rg97n29	Shake	Today, 02:37:00	2s
Shake.6rg97hja	Shake	Today, 02:36:55	2s
Shake.6rg97ets	Shake	Today, 02:36:52	2s
Shake.6rg97cah	Shake	Today, 02:36:49	2s

Buttons: Collect data Connect a device to start building your dataset.

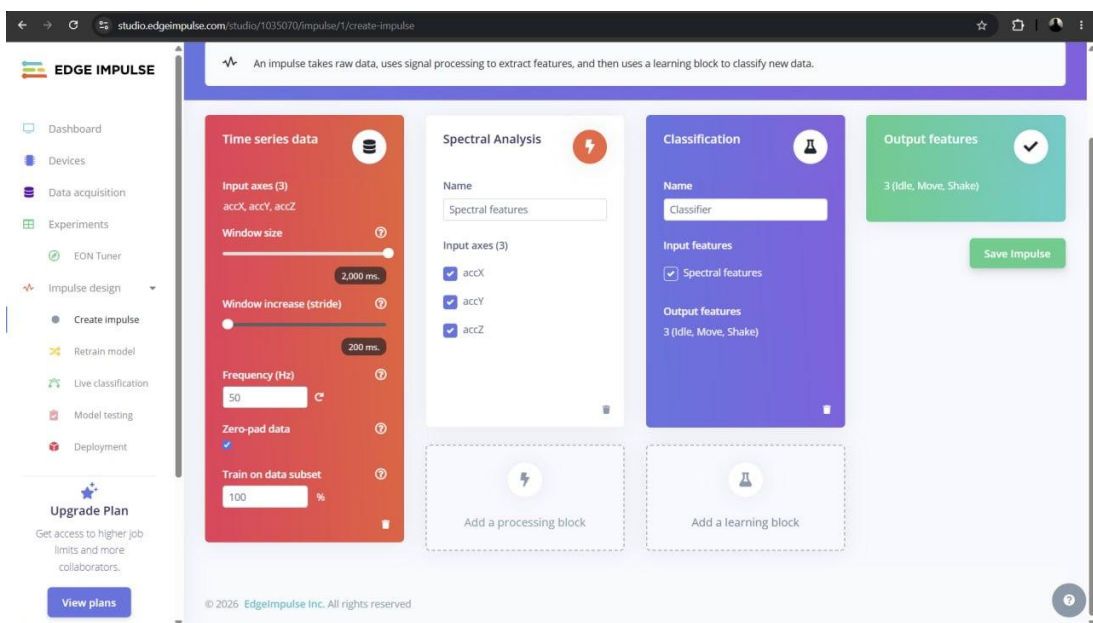
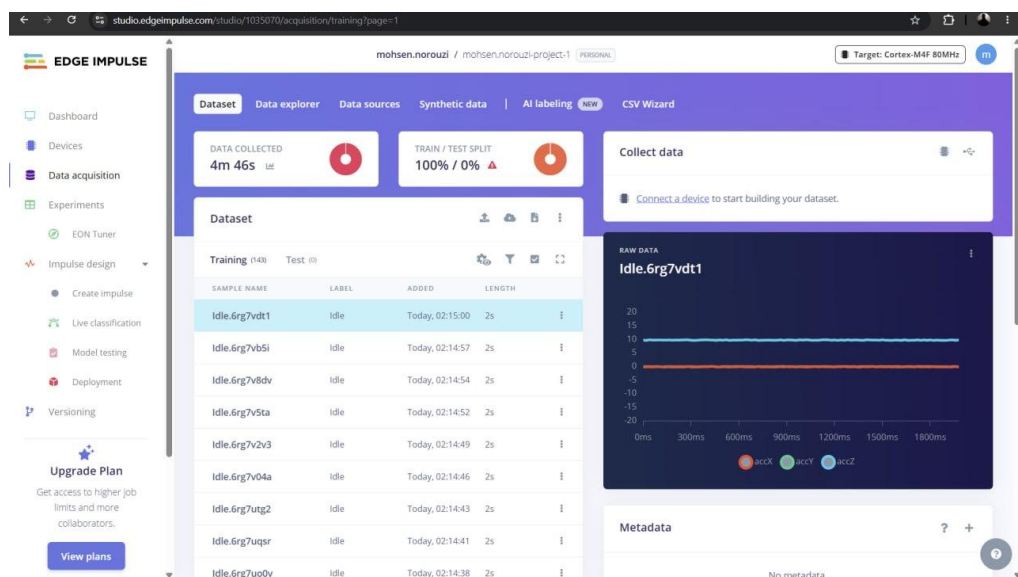
RAW DATA: Click on a sample to load...

8-طراحی Impulse و استخراج ویژگی‌ها

در مرحله بعد ساختار Impulse طراحی شد.

تنظیمات استفاده شده:

- Window Size = 2000ms
 - Processing Block = Spectral Features
 - Learning Block = Classification (Keras)
- پس از طراحی Impulse عملیات Generate Features انجام شد.



9-آموزش و ارزیابی مدل

برای بررسی عملکرد مدل دو تنظیم مختلف آزمایش شد.

پیکربندی اول

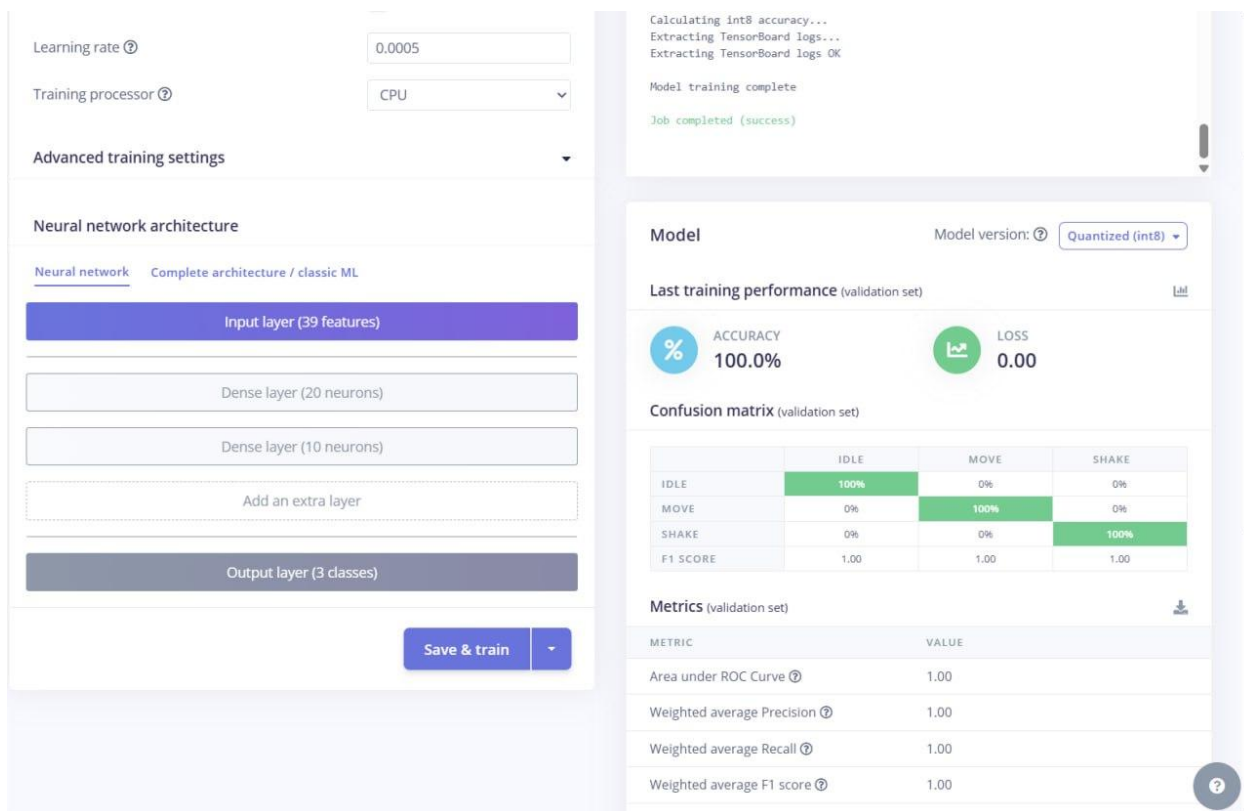
- Window Size = 2000ms
- تنظیمات پیش فرض Classifier

نتایج:

- Accuracy
- Confusion Matrix ثبت شد.



«در هر دو تنظیم (پنجره ۲۰۰۰ و ۱۰۰۰ میلی‌ثانیه)، دقت مدل ۱۰۰٪ به دست آمد. دلیل این امر، استفاده از داده‌های شبیه‌سازی‌شده (Synthetic) و توابع قطعی ریاضی (مانند موج سینوسی برای حرکت و نویز مقیاس‌دار برای لرزش) است. به دلیل عدم وجود نویزهای فیزیکی سنسور و تفکیک‌پذیری کامل کلاس‌ها در فضای ویژگی‌ها (Feature Space)، مدل توانست حتی در پنجره‌های زمانی کوتاه‌تر نیز بدون خطا الگوها را دسته‌بندی کند.»



10-استقرار مدل (Deployment)

پس از انتخاب مدل مناسب، مدل به صورت نسخه سبک TinyML مستقر شد.

نوع: Deployment

• Linux x86

یا

• Python Library

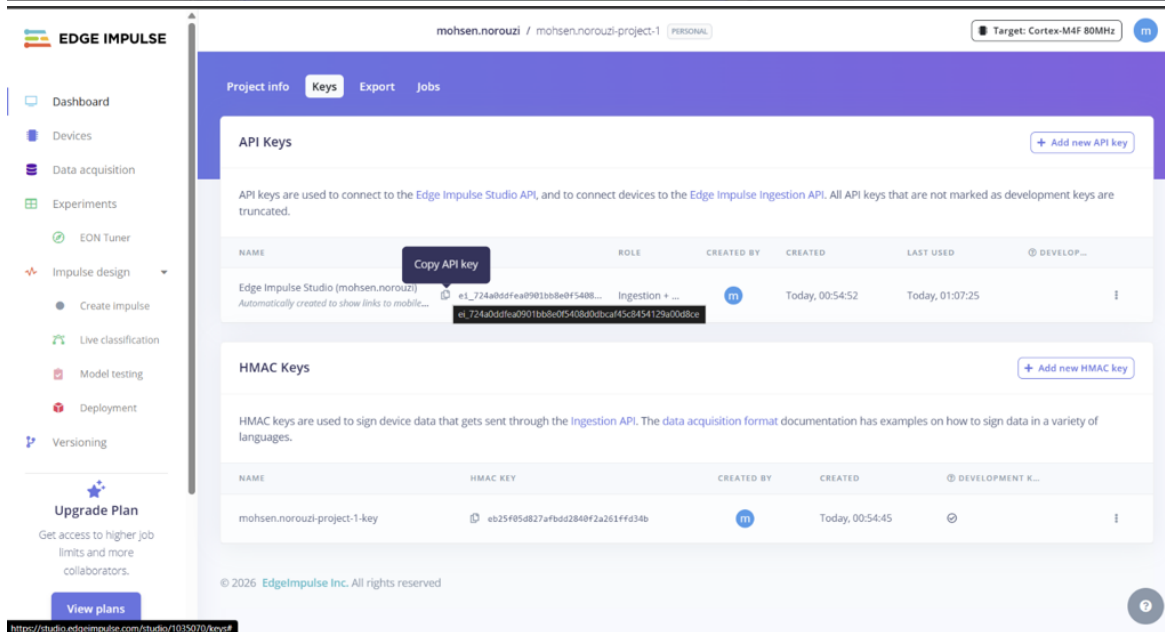
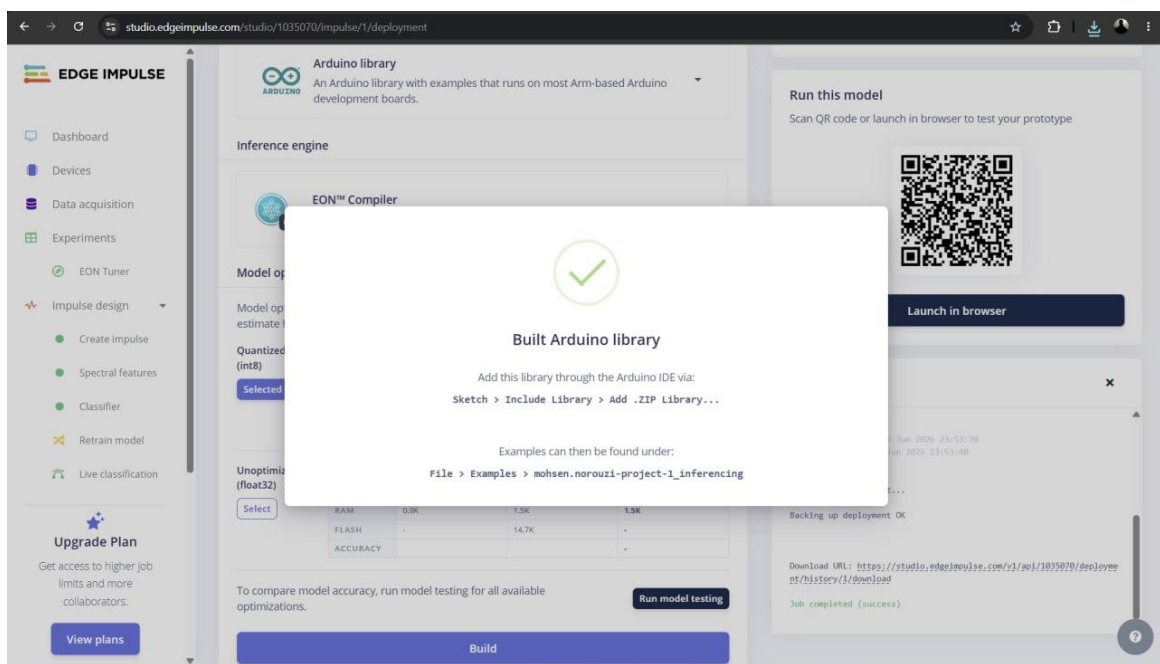
مدل آموزش‌دیده برای انجام Inference بر روی داده‌های جدید استفاده گردید.

11- اجرای برخط و انجام Inference

در این مرحله داده‌های جدید تولید شده توسط Velxio وارد مدل شدند.

مدل به صورت لحظه‌ای وضعیت حرکت را تشخیص داد:

- Idle
- Move
- Shake





12- اتصال به ThingsBoard

یک Device جدید در ThingsBoard ایجاد شد.

خروجی مدل در قالب Telemetry ثبت گردید.

کلیدهای اصلی:

- state
- confidence
- timestamp

The screenshot shows the ThingsBoard web interface. On the left is a sidebar with navigation options like Home, Alarms, Dashboards, Entities, Devices, Assets, Entity views, Gateways, Profiles, Customers, Rule chains, Edge management, Advanced features, Resources, Notification center, Mobile center, API usage, Settings, and Security. The main area is divided into two panels. The left panel shows a list of devices with columns for 'Created time', 'Name', and 'Device profile'. The right panel shows the details for the 'AI_Motion_Detector' device, including tabs for 'Details', 'Attributes', 'Latest telemetry', 'Calculated fields', 'Alarms', 'Events', and 'Relations'. The 'Latest telemetry' tab is active, displaying a table with columns for 'Last update time', 'Key', and 'Value'. The table contains two entries: one for 'confidence' with a value of '0.99609375' and one for 'motion_state' with a value of 'Shake'.

The screenshot shows the ThingsBoard web interface with the 'AI Output Dashboard' selected. A modal dialog box titled 'Add alias' is open in the center. The dialog has fields for 'Alias name*' (containing 'MySensor'), 'Filter type*' (set to 'Single entity'), 'Type*' (set to 'Device'), and 'Device*' (containing 'AI_Motion_Detector'). There is a checkbox for 'Resolve as multiple entities' which is currently unchecked. At the bottom of the dialog are 'Cancel' and 'Add' buttons. The background shows the dashboard layout with various widgets and a sidebar on the left.

13- پیاده سازی سیستم هشدار

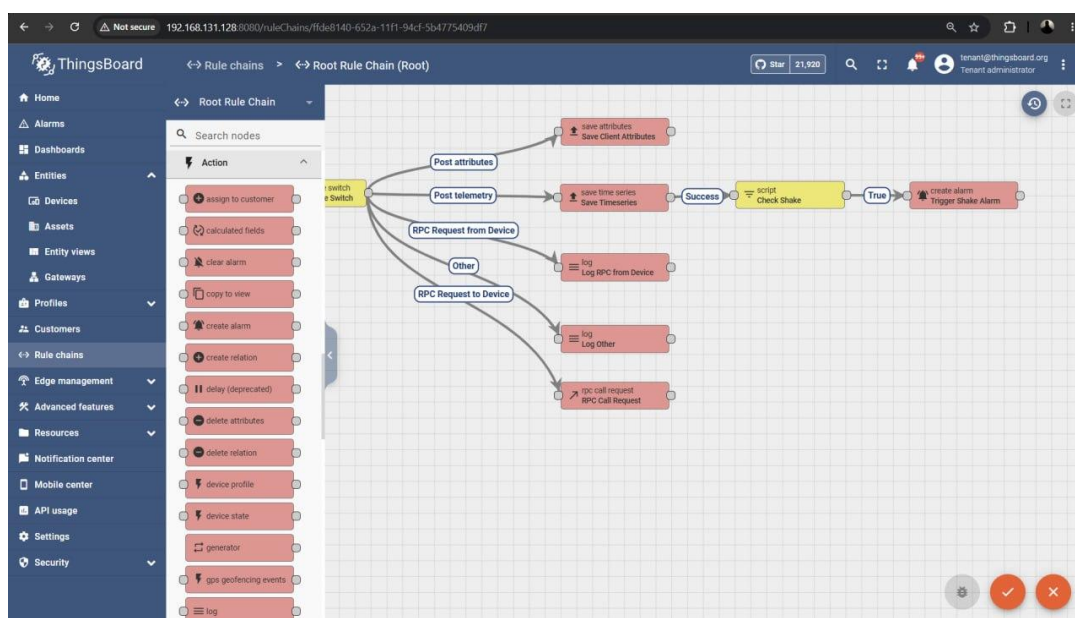
برای تشخیص وضعیت خطرناک Shake از Rule Chain استفاده شد.

منطق هشدار:

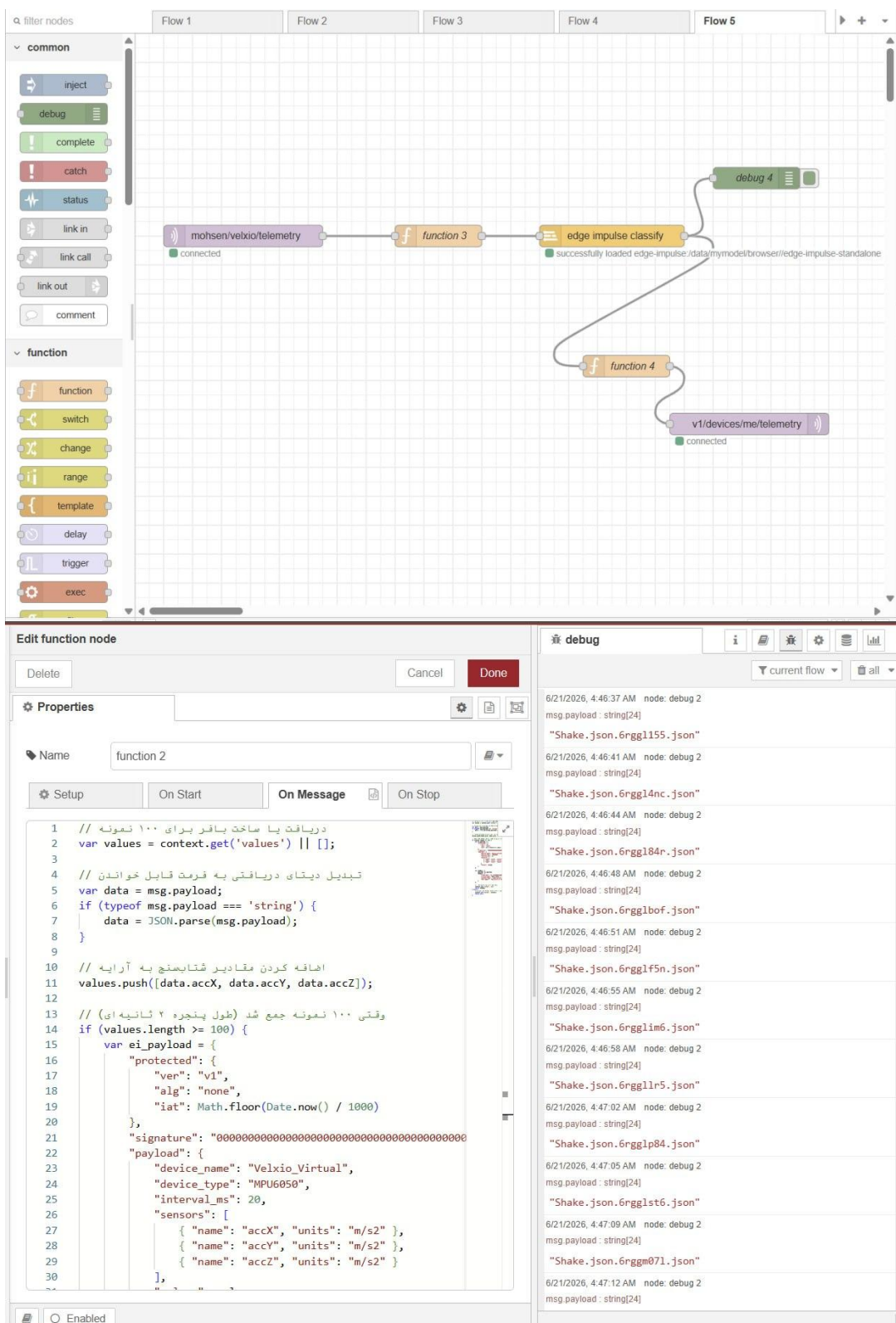
`return msg.state == "Shake";`

در صورت وقوع Shake:

- Alarm ایجاد می شود.
- هشدار در داشبورد نمایش داده می شود.



14- اضافه کردن مسیر به node-red

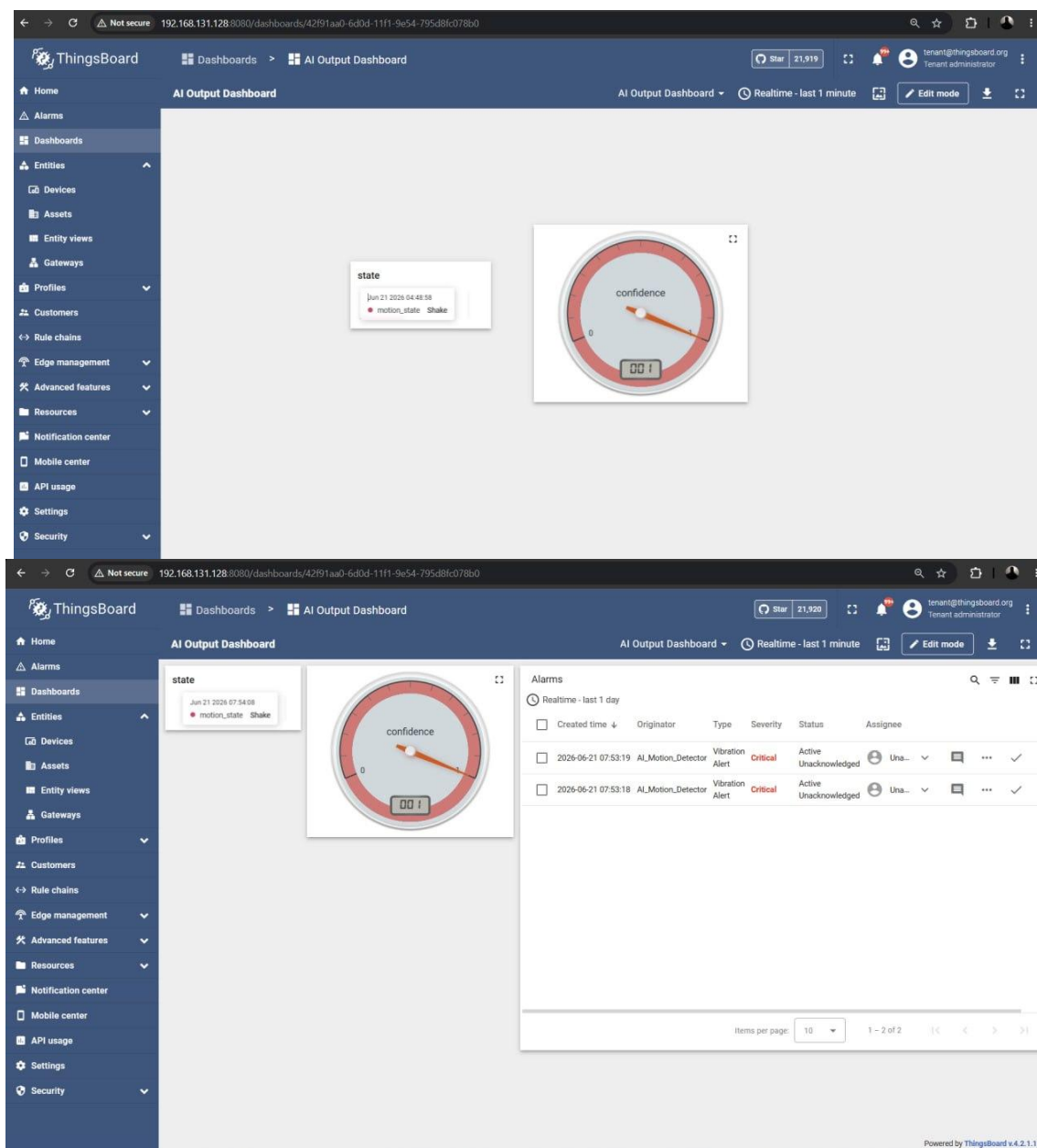


15- داشبورد نهایی

برای نمایش وضعیت سیستم یک داشبورد طراحی شد. اجزای داشبورد:

- وضعیت فعلی حرکت
- نمودار تغییرات
- Confidence مدل
- لیست هشدارها

در صورت تشخیص Shake هشدار به صورت زنده نمایش داده می‌شود.





نتیجه گیری

در این پروژه یک زنجیره کامل AIoT شامل شبیه سازی حسگر، جمع آوری داده، آموزش مدل TinyML، استقرار در لبه و نمایش نتایج در ThingsBoard پیاده سازی شد.

نتایج نشان دادند که مدل های سبک Edge Impulse قابلیت مناسبی برای تشخیص وضعیت های حرکتی دارند و می توانند بدون نیاز به پردازش ابری، تصمیم گیری را در لبه شبکه انجام دهند. استفاده از TinyML موجب کاهش تأخیر، کاهش مصرف پهنای باند و افزایش سرعت پاسخ گویی سامانه گردید.